Lecture 6

Searching Algorithms: Linear VS Binary

Searching Algorithms: Linear Search VS Binary Search

Part 1: Mastering Linear Search

Concept, Algorithm, and Complexity Analysis

© 2026 Ahmed Elrefaiy

The Idea: The Sequential Walk

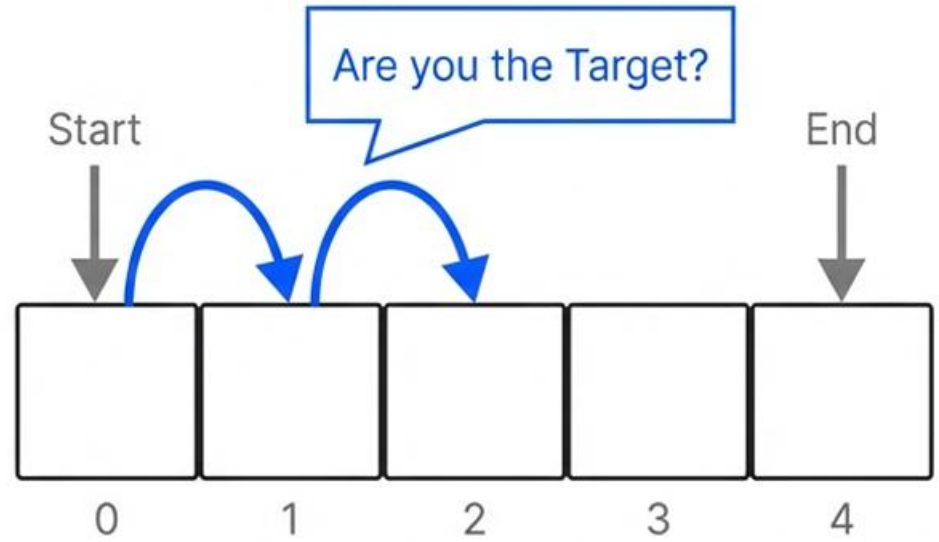
Linear Search is a foundational algorithm used to find a specific item (the "Key") within a larger collection (an "Array").

The Starting Point: Begin at the very first element (Index 0).

The Core Question: Ask the current element: "Are you the Key?"

The Action:
- If Yes -> Stop searching. The item is found.
- If No -> Move exactly one step forward.

The Conclusion: If the end of the array is reached and the answer was always "No", the item does not exist.



It is the most straightforward search method—checking every single possibility systematically until the answer is found or options are exhausted.

Example 1: Finding the Target (Success)

`Array: [10 | 50 | 30 | 20 | 70 | 80 | 60 | … | 100]`

`Key (Target): 20`

Check 10. Is it 20? No -> Move to next.

Step 1	10	50	30	20	70	80	60	…	100

Check 50. Is it 20? No -> Move to next.

Step 2	10	50	30	20	70	80	60	…	100

Check 30. Is it 20? No -> Move to next.

Step 3	10	50	30	20	70	80	60	…	100

Check 20. Is it 20? YES -> Stop!

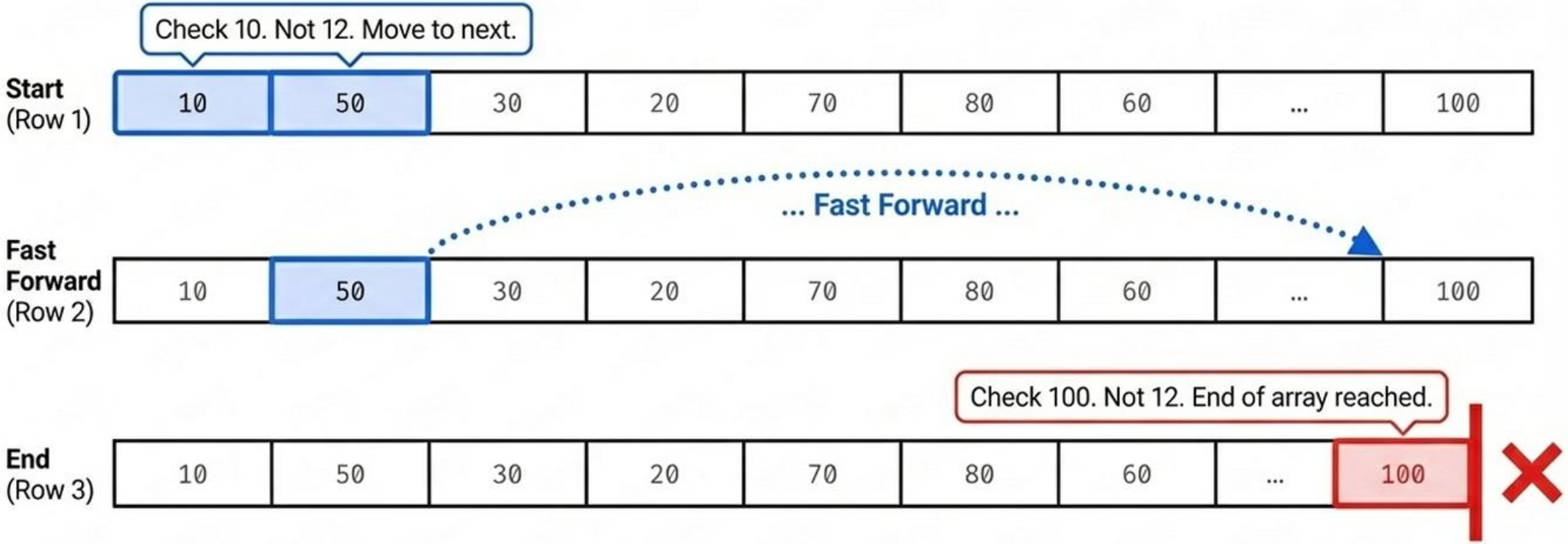
Step 4	10	50	30	20	70	80	60	…	100

Search Terminated Successfully. The algorithm stops immediately upon finding the target, **saving unnecessary future checks.**

Example 2: The Exhaustive Search (Not Found)

`Array: [10 | 50 | 30 | 20 | 70 | 80 | 60 | … | 100]`

`Key (Target): 12`



Search Exhausted. The pointer has stepped through every single index. With no remaining numbers to check, the algorithm confidently concludes the Key is absent.

The Algorithm: Two Implementations

Required: Array (arr), Length (n), Target (key).
Loop runs from i = 0 to i <= n - 1.

Version A: Return Boolean	Version B: Return Index

`function linearSearch(arr, n, key):`
`  for i from 0 to n-1:`
`  if arr[i] == key:`
`  return true` ← Immediate Exit.
 	
`  return false` ← Loop finished, not found.
	`function linearSearch(arr, n, key):`
	`  for i from 0 to n-1:`
	`  if arr[i] == key:`
	`  return i` ← Returns exact memory location.
	
	`  return -1` ← -1 indicates a missing item.

The Power of **return**: The success trigger is inside the loop (stopping execution instantly), while the failure trigger is outside (only executing if all checks fail).

Time Complexity: Analyzing Efficiency

`Based on the equation T(n) = Sum(1 check)`

Best Case: Ωmega(1)

Scenario: The Key is found at the very first index (Index 0).
Logic: Only 1 step is executed. The algorithm immediately exits.

Average Case: Theta(n)

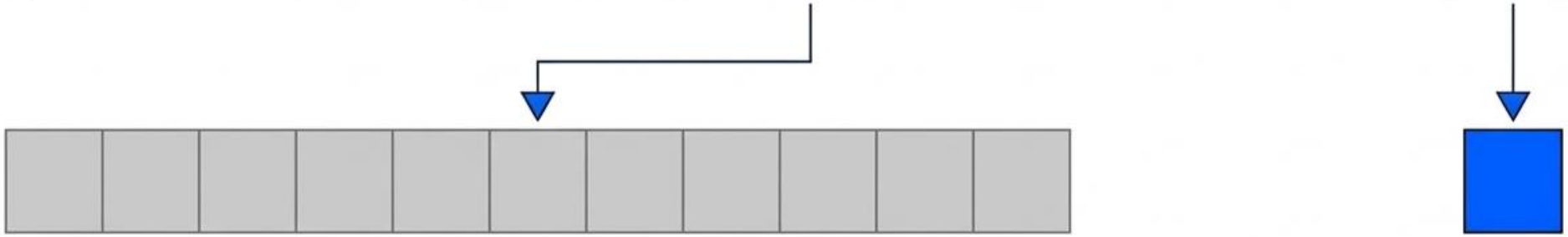
Scenario: The Key is found somewhere in the middle of the array.
Logic: It takes approximately n/2 steps. In complexity notation, we drop mathematical constants (like 1/2), simplifying the strict bound to Theta(n).

Worst Case: O(n)

Scenario: The Key is at the very last index, or not in the array at all.
Logic: The loop executes its maximum number of times (n steps). Performance scales directly linearly with the size of the array.

Space Complexity: Memory Efficiency

[Total Space] = [Input Space] + [Auxiliary Space]



The memory required to hold the array.
Fixed across different search algorithms.

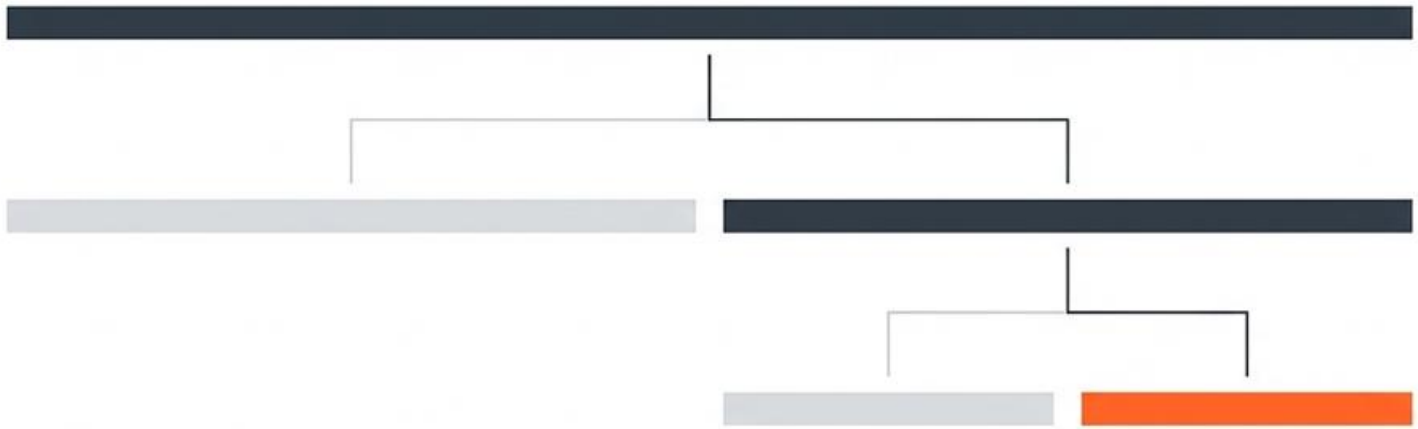
The extra memory required by the algorithm to do its job. We focus our analysis here.

Application to Linear Search

- How many new variables were created to run the search? Only one integer variable is declared: the index tracker (`int i = 0`).
- Whether the array has 10 items or 10 million items, the algorithm only ever needs that single extra variable to track its position.

Verdict: O(1) Constant Space

Linear search has a strictly constant memory footprint.



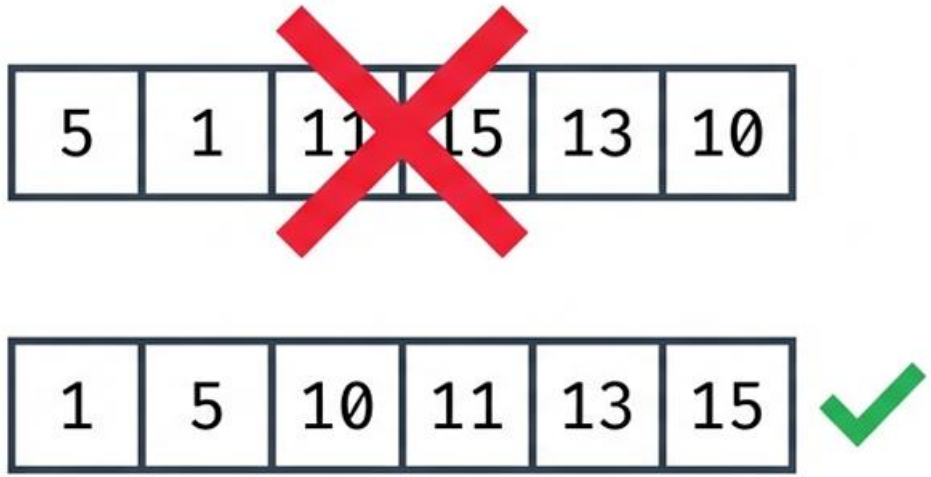
Searching Algorithms: 2. Binary Search

A deep dive into logarithmic search mechanics, complexity analysis, and implementation.

[Recursive & Iterative Versions]

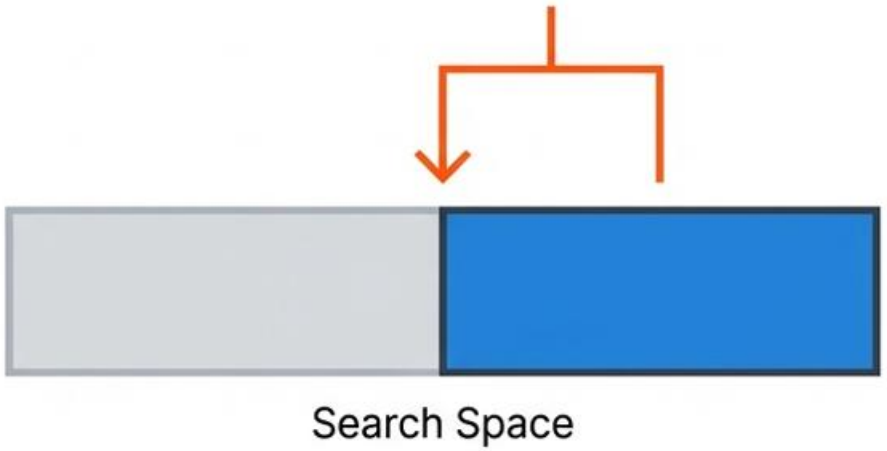
The Idea: Divide, Conquer, and Eliminate

The Prerequisite



THE PREREQUISITE: The array must be strictly sorted. If unsorted, you must apply a sorting algorithm before searching.

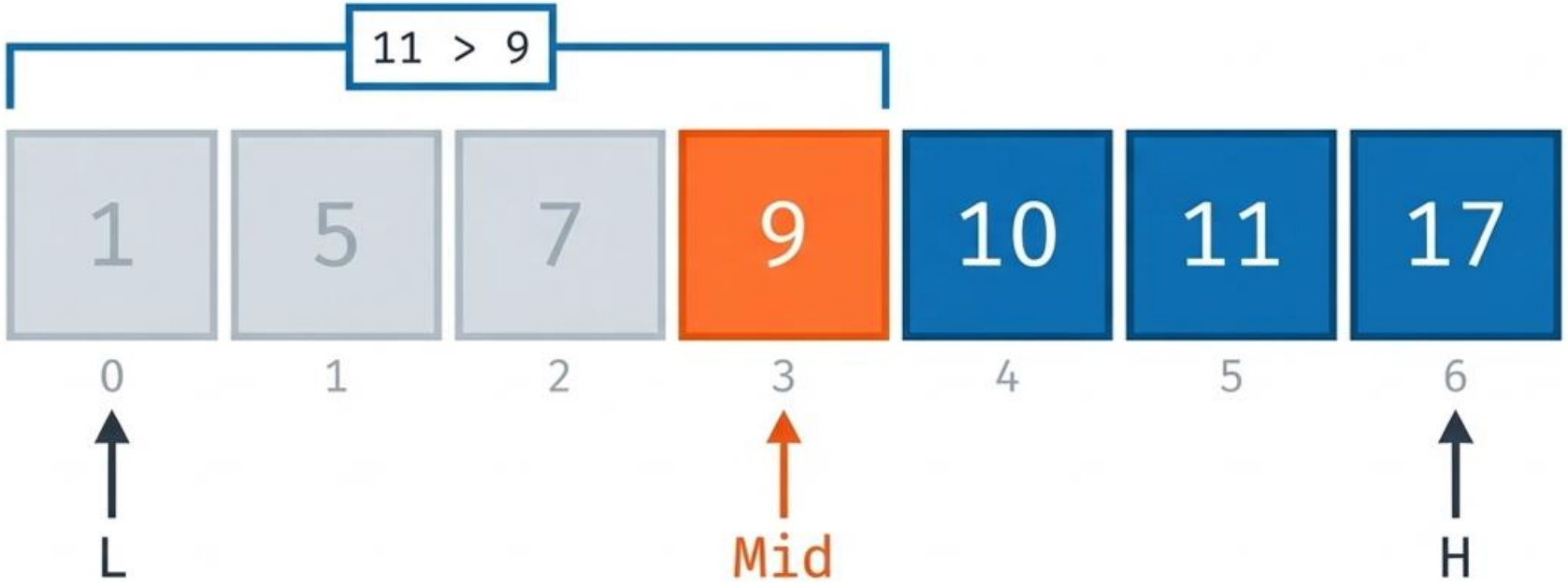
The Core Idea



Instead of checking elements one by one (Linear Search), Binary Search eliminates half of the remaining search space with every single step.

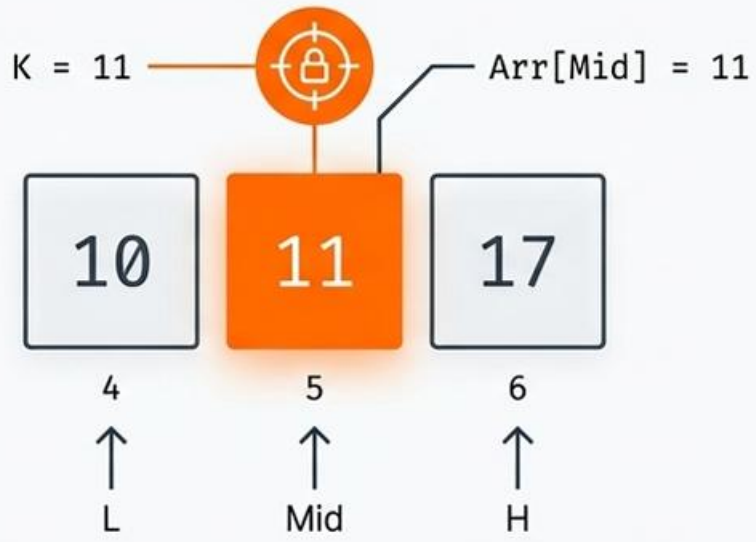
Setup & The First Halving

Target: K = 11



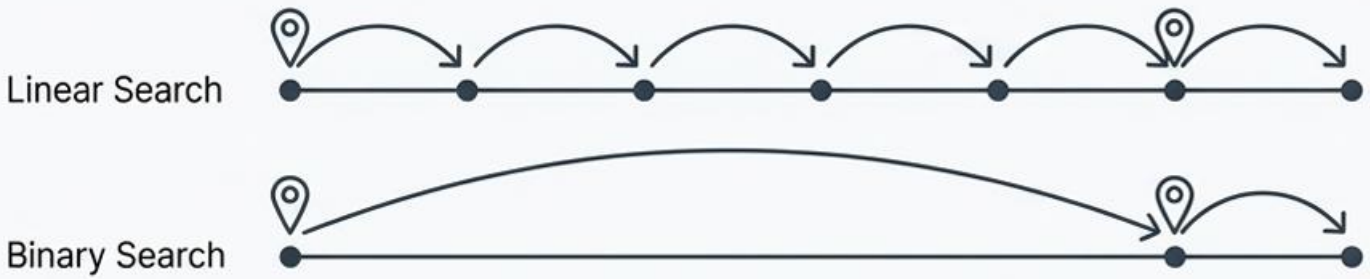
Step 1: Define boundaries (L=0, H=6).
Step 2: Calculate Midpoint (Mid = 3 -> Value is 9).
Step 3: Compare. 11 > 9.
Action: Target is greater than the midpoint. Discard the entire left half.

Resolution & Efficiency



Step 4: New boundaries (L=4, H=6).
New Midpoint
(Mid = 5 —> Value is 11).

Step 5: Compare.
11 == 11.
Target Found.



Insight: To find 11 in this array, Linear Search requires 6 steps.

Binary Search required exactly 2 steps.

The Recursive Implementation

```
BinarySearch(Arr, L, H, K):
  if (L > H) return -1
  Mid = L + (H - L) / 2
  if (Arr[Mid] == K) return Mid
  if (K > Arr[Mid])
     return BinarySearch(Arr, Mid+1, H, K)
  else
     return BinarySearch(Arr, L, Mid-1, K)
```

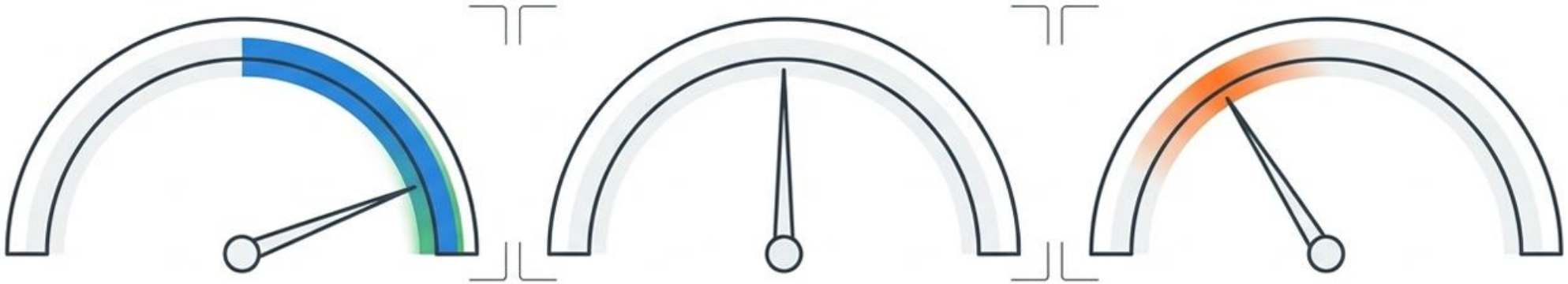
Base Case: Search space exhausted. Target not found.

The Pivot: Calculates midpoint while preventing standard integer overflow.

The Recursion: Pass updated boundaries (Mid+1 or Mid-1) to search the remaining half.

Recursive Time Complexity

$$T(n) = T(n/2) + 1 \rightarrow O(\log n)$$



`Omega(1)`

Best Case: Target is found at the very first midpoint calculated.

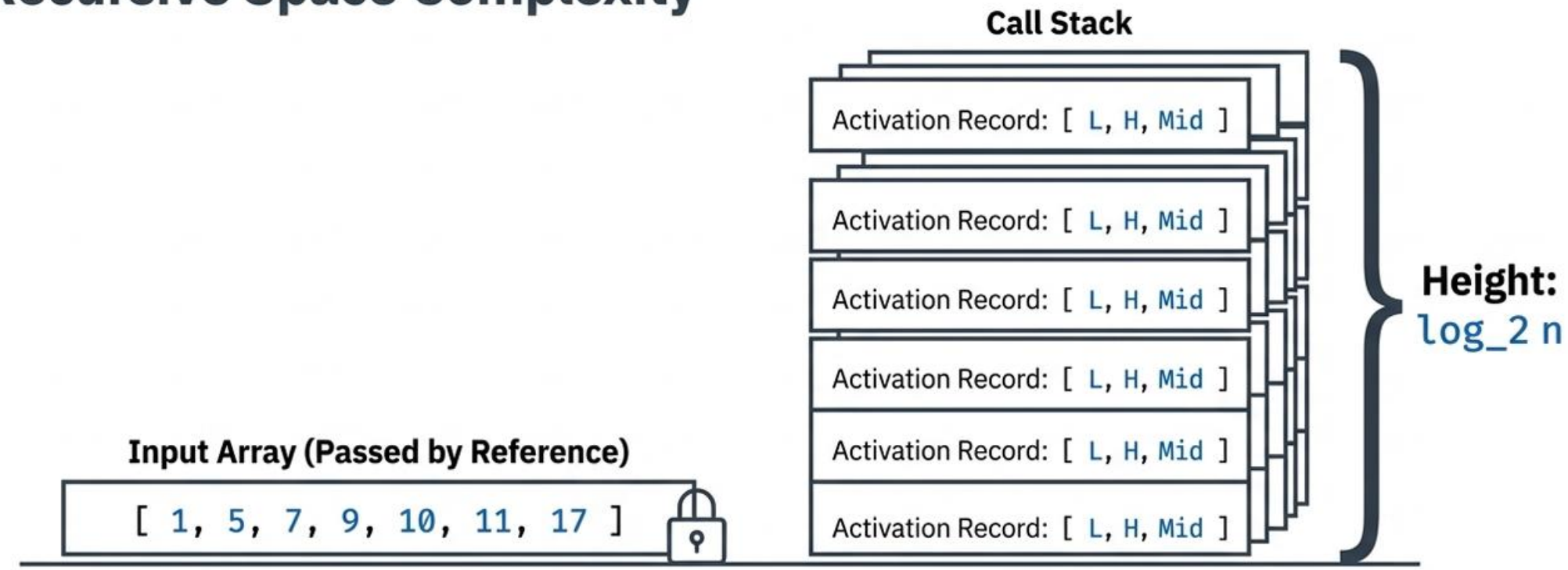
`Theta(log n)`

Average: Target found deep in the division tree.

`O(log n)`

Worst: Target is at a leaf node or not present (halving until L > H).

Recursive Space Complexity



Input Space: Fixed.
The array is not duplicated.

Auxiliary Space: O(log n)

Why? Every recursive call waits in memory for the next to resolve. For an array of size N, the maximum depth of the call stack tree is log_2 n, accumulating local variables at each level.

The Showdown: Recursive vs. Iterative

Recursive Approach

```
int BinarySearch(int Arr[], int L, int H, int K) {
    if (L > H) return -1; // Base case: not found
    int Mid = L + (H - L) / 2;
    if (Arr[Mid] == K) return Mid; // Base case: found
    if (K > Arr[Mid]) {
        return BinarySearch(Arr, Mid+1, H, K); // Recursive call
    } else {
        return BinarySearch(Arr, L, Mid-1, K); // Recursive call
    }
}
```

Relies on the compiler's call stack to pass updated L and H boundaries via new function calls.

Iterative Approach

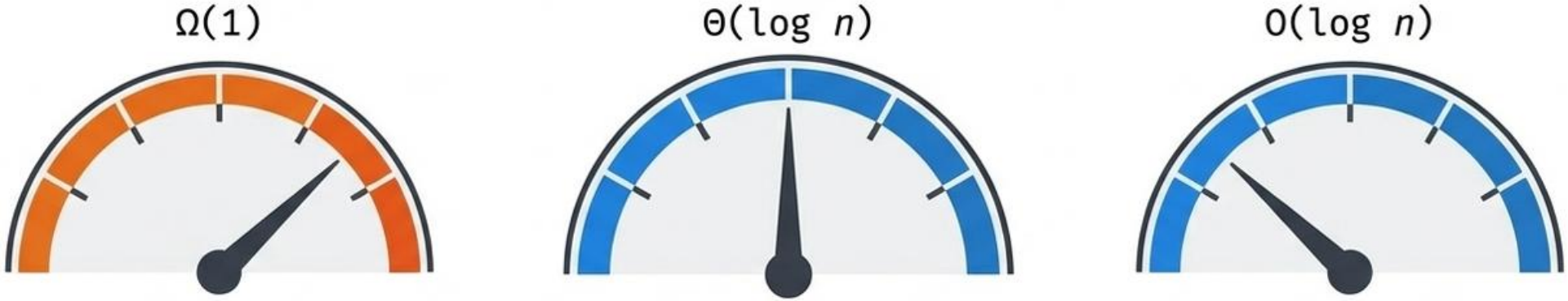
```
int BinarySearch(int Arr[], int N, int K) {
    int L = 0, H = N - 1;
    while (L <= H) {
        int Mid = L + (H - L) / 2;
        if (Arr[Mid] == K) return Mid; // Found
        if (K > Arr[Mid]) {
            L = Mid + 1;
        } else {
            H = Mid - 1;
        }
    }
    return -1; // Not found
}
```

Utilizes a single while loop, manually overwriting the L and H pointers in memory.

Synthesis: The mathematical halving is identical; the difference is purely structural architecture.

Iterative Time Complexity

Optimized:
Zero Call Overhead.



The execution speed remains mathematically identical to the recursive version. The number of comparisons is exactly the same, and the active search space is still strictly halved per iteration of the while loop.

Real-World Edge: While Big-O is identical, the iterative version is practically faster in production environments due to the complete absence of function call overhead and stack allocation.

Iterative vs. Recursive Space Complexity Comparison

Recursive Approach (O(log n))

Requires **new stack frame for each recursive call**, leading to **logarithmic space usage** as the call stack depth increases.

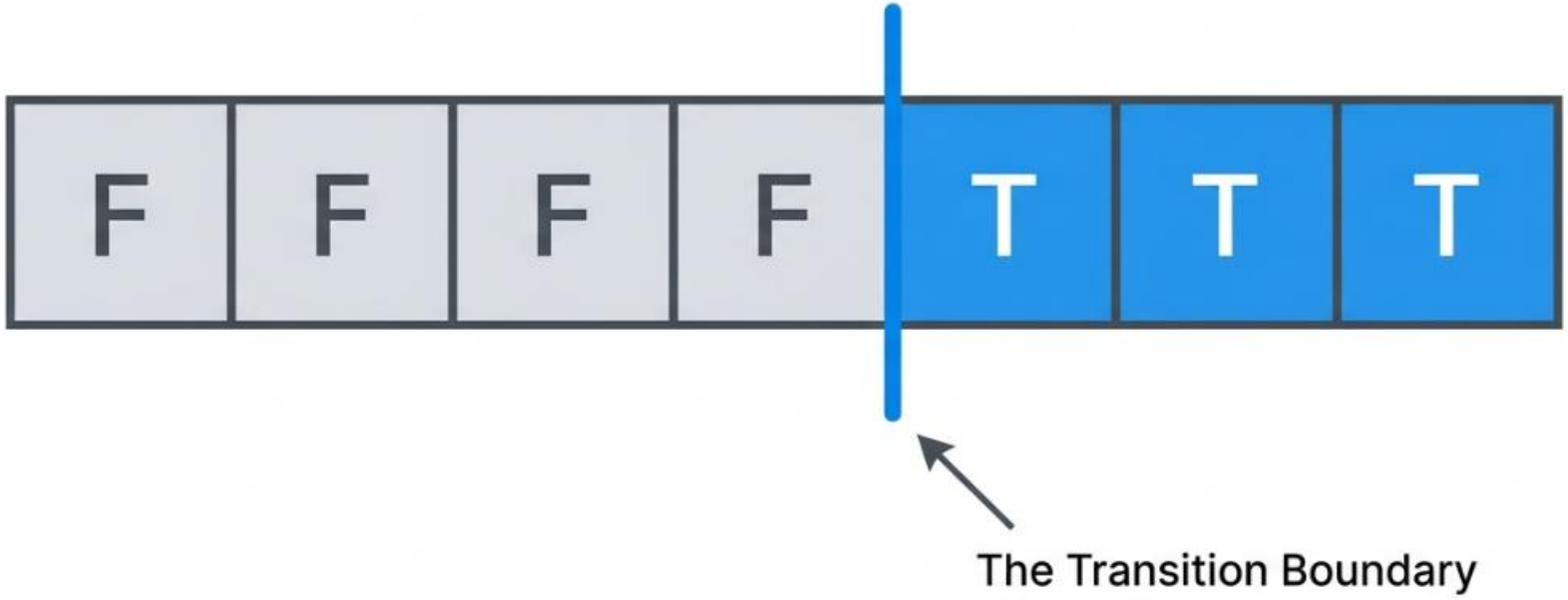
Iterative Approach (O(1))

The architectural shift eliminates the call stack entirely. The iterative algorithm requires **only a few pointers** (**`L, H, Mid`**) which are **continually overwritten** in a **single memory block.** The spatial footprint remains perfectly flat, regardless of how massive the input array becomes.

Synthesis: Iterative approach offers significant memory advantages, especially for large inputs, by **eliminating** the call stack overhead.

Problems Solved by Binary Search

Moving beyond simple lookups to master lower bounds, upper bounds, and monotonic transitions.



The Fundamental Prerequisite is Monotonicity

Binary search solves problems by finding the exact position where a condition becomes True. This only works if the checking condition applied to the array is **Monotonic**—meaning all False results are grouped together, and all True results are grouped together, featuring exactly one transition.

The Monotonic Success

Data Array:	1	2	3	3	2	5	6

Condition applied: Is the item >= 4?

Boolean Array:	F	F	F	F	F	T	T

The Transition: Happens exactly once.

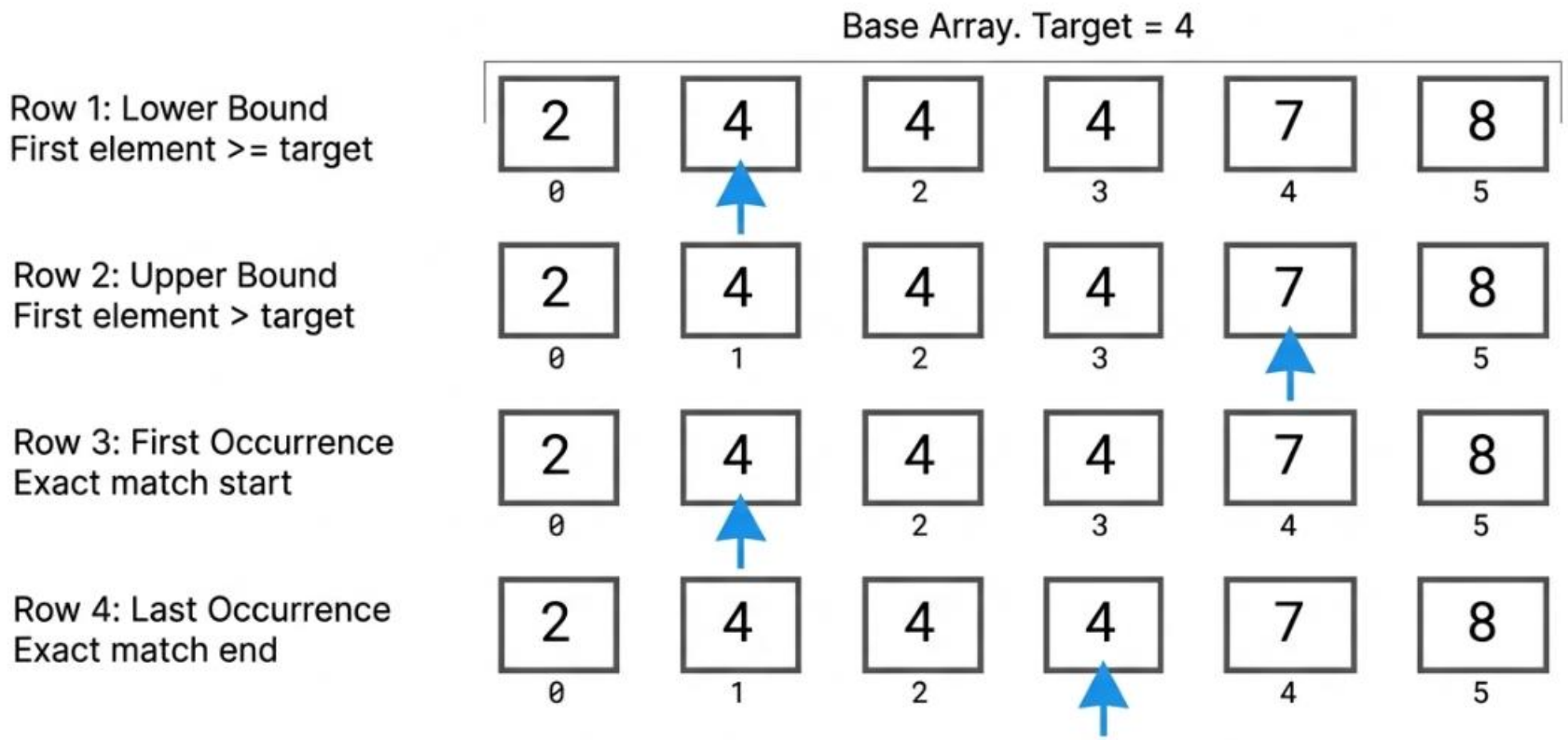
The Counter-Example

Boolean Array:	T	F	T	F	F	T

Not Monotonic.
Multiple transitions exist.
Binary search fails here.

One Monotonic Concept Yields Four Classic Operations

If an array is sorted, we can leverage the monotonic property to solve four distinct boundary problems.

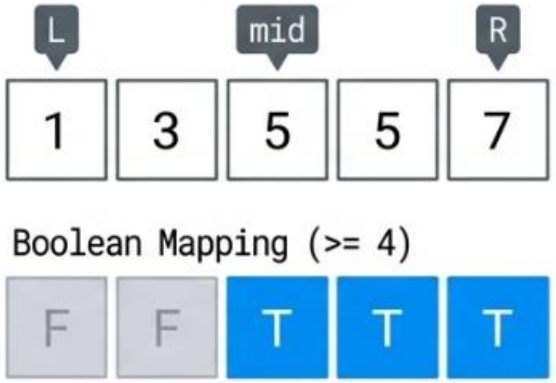


Finding the Lower Bound Insertion Point

Locating the index of the very first element that is >= target.

```
LowerBound(arr, L, R, key):
  while L < R:
    mid = L + (R - L) / 2
    if arr[mid] >= key:
      R = mid  // Target is at mid or left of mid
    else:
      L = mid + 1 // Target is strictly to the right
  return L
```

Array: [1, 3, 5, 5, 7] | Target = 4



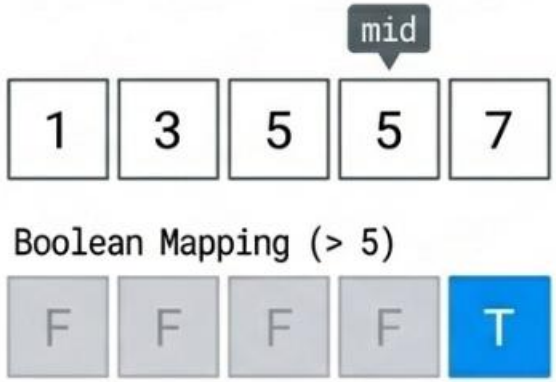
Why R = mid? If arr[mid] >= key, the current element might be the actual boundary. We cannot use mid - 1 because we would discard the correct answer.

Finding the Upper Bound Limit

Locating the index of the first element that is strictly > target.

```
UpperBound(arr, L, R, key):
  while L < R:
    mid = L + (R - L) / 2
    if arr[mid] > key:
      R = mid  // Target is at mid or left of mid
    else:
      L = mid + 1 // Target is at mid or to the right
  return L
```

Array: [1, 3, 5, 5, 7] | Target = 5



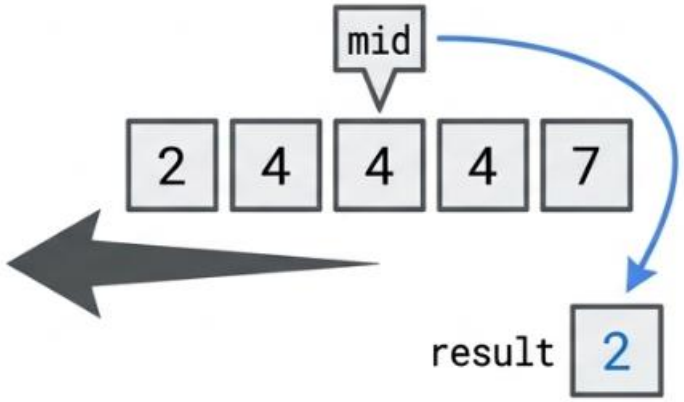
Notice the shift. We only look for strictly greater values. Even if we hit the target exactly (the 5), it evaluates to False, forcing the search to continue to the right (L = mid + 1).

Locating the First Occurrence of a Target

Finding the exact starting index of a repeating sequence.

```
FindFirstOccurrence(arr, L, R, key):
  result = -1
  while L <= R:
    mid = L + (R - L) / 2
    if arr[mid] == key:
      result = mid  // Record it,
    but keep searching left!
      R = mid - 1
    elif arr[mid] < key: L = mid + 1
    else: R = mid - 1
  return result
```

Array: [2, 4, 4, 4, 7] | Target = 4



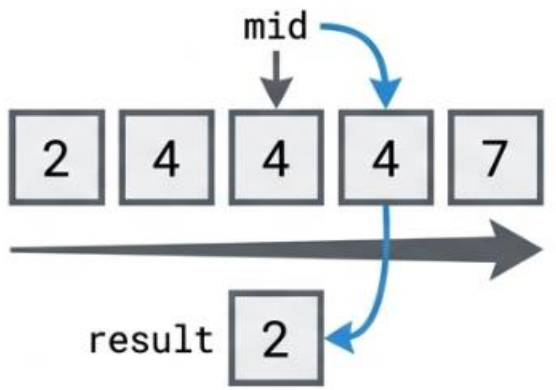
When we find a match, we save it. But we don't stop. We force **R = mid - 1** to check the left side to see if there is an earlier occurrence.

Locating the Last Occurrence of a Target

Finding the exact ending index of a repeating sequence.

```
FindLastOccurrence(arr, L, R, key):
  result = -1
  while L <= R:
    mid = L + (R - L) / 2
    if arr[mid] == key:
      result = mid   // Record it, but
    keep searching right!
      L = mid + 1
    elif arr[mid] < key: L = mid + 1
    else: R = mid - 1
  return result
```

Array: [2, 4, 4, 4, 7] | Target = 4



The mirror image of First Occurrence. We save the match, but force L = mid + 1 to check the right side to see if the sequence continues further.

The Binary Search Diagnostic Test

Not sure if a complex problem can be optimized? Run it through this two-step diagnostic flowchart.

Step 1: Can you convert the problem into a True/False question for each individual item?

↓ **Yes**

Step 2: Is the resulting T/F sequence Monotonic? (Are all Fs grouped together, and all Ts grouped together, with only ONE transition?)

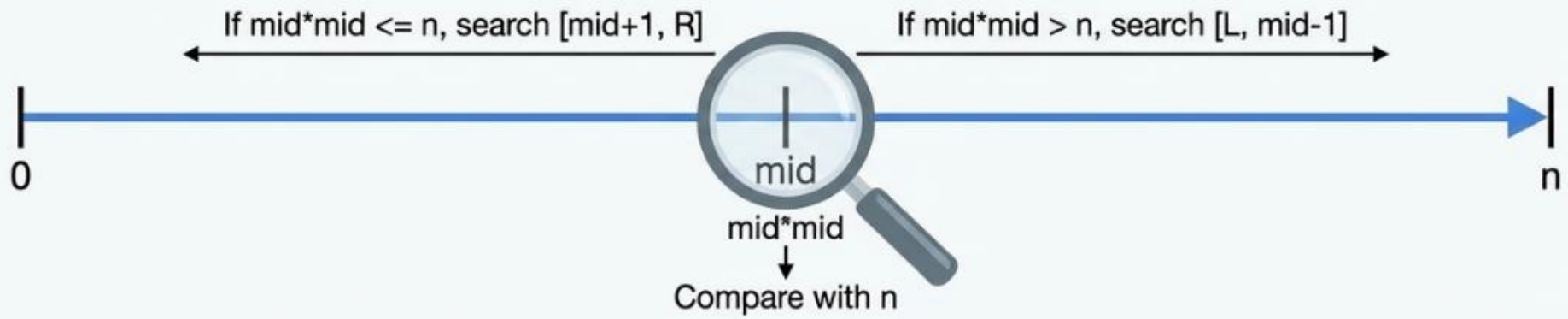
↓ **Yes**

Binary Search Will Work!

If your condition maps to an array like [F, F, F, T, T, T], you can use **Binary Search** to find that exact transition point in O(log n) time.

Advanced Task: Find Square Root of n using Binary Search on the Answer Space

The idea: The square root of ‘n’ must lie within the range [0, n]. We can binary search this range to find the largest integer ‘x’ such that x*x <= n.



The Range

The answer space is the continuous sorted range [0, n].

The Condition

We define a custom function `isValid(mid)` which checks if `mid * mid <= n`. This returns True or False.

The Logic

If `isValid(mid)` is True, the square root is at `mid` or to its right. We store `mid` and continue searching the right half. If False, we search the left half.

Questions

Choose Correct Answer

- Number of steps to find **element 11** using **<u>linear search</u>** =
  a. 2
  b. 4
  c. 5
  d. 6
- Number of steps to find **element 11** using **<u>Binary search</u>** =
  a. 2
  b. 4
  c. 5
  d. 6

Choose Correct Answer

- Number of steps to find **element 17** using **<u>linear search</u>** =
  a. 3
  b. 5
  c. 6
  d. 7
- Number of steps to find **element 17** using **<u>Binary search</u>** =
  a. 2
  b. 3
  c. 6
  d. 7

Answer the following questions

- Worst case of Linear search = ……
- Best case of Linear search = ……
- Average case of Linear search = ……
- Space complexity for Linear search = ……

- Worst case of Binary search (recursive version) = ……
- Best case of Binary search (recursive version) = ……
- Average case of Binary search (recursive version) = ……

- Worst case of Binary search (Iterative version) = ……
- Best case of Binary search (Iterative version) = ……
- Average case of Binary search (Iterative version) = ……

- Space complexity for Binary search (Iterative version) = ……
- Space complexity for Binary search (recursive version) = ……

Write and implement the following Algorithms

- Linear search
- Binary search (iterative version)
- Binary search (recursive version)
- Write **recursive version** algorithm of **Linear search**? And find its Time and Space complexity?

Write and Implement the following algorithms

1. Find (Lower Bound) the index of the first element >= target (insertion point).
2. Find (Upper Bound) the index of the first element > target.
3. Find the index of the first occurrence of target
4. Find the index of the last occurrence of target?

5. **[Your Task]** write algorithm and implement the following algorithm:
   - **Find square root of n using binary search on the answer space**